

Quick-Start: Regex Cheat Sheet

PROTECTED BY **COPYSCAPE** DO NOT COPY

The tables below are a reference to basic regex. While reading the rest of the site, when in doubt, you can always come back and look here. (If you want a bookmark, here's a direct link to the [regex reference tables](#)). I encourage you to print the tables so you have a cheat sheet on your desk for quick reference.

The tables are not exhaustive, for two reasons. First, every regex flavor is different, and I didn't want to crowd the page with overly exotic syntax. For a full reference to the particular regex flavors you'll be using, it's always best to go straight to the source. In fact, for some regex engines (such as Perl, PCRE, Java and .NET) you may want to check once a year, as their creators often introduce new features.

The other reason the tables are not exhaustive is that I wanted them to serve as a quick introduction to regex. If you are a complete beginner, you should get a firm grasp of basic regex syntax just by reading the examples in the tables. I tried to introduce features in a logical order and to keep out oddities that I've never seen in actual use, such as the "bell character". With these tables as a jumping board, you will be able to advance to mastery by exploring the other pages on the site.

How to use the tables

The tables are meant to serve as an accelerated regex course, and they are meant to be read slowly, one line at a time. On each line, in the leftmost column, you will find a new element of regex syntax. The next column, "Legend", explains what the element means (or encodes) in the regex syntax. The next two columns work hand in hand: the "Example" column gives a valid regular expression that uses the element, and the "Sample Match" column presents a text string that could be matched by the regular expression.

You can read the tables online, of course, but if you suffer from even the mildest case of online-ADD (attention deficit disorder), like most of us... Well then, I highly recommend you print them out. You'll be able to study them slowly, and to use them as a cheat sheet later, when you are reading the rest of the site or experimenting with your own regular expressions.

Enjoy!

If you overdose, make sure not to miss the next page, which comes back down to Earth and talks about some really cool stuff: **[The 1001 ways to use Regex](#)**.

Regex Accelerated Course and Cheat Sheet

For easy navigation, here are some jumping points to various sections of the page:

- * [Characters](#)
- * [Quantifiers](#)
- * [More Characters](#)
- * [Logic](#)
- * [More White-Space](#)
- * [More Quantifiers](#)
- * [Character Classes](#)
- * [Anchors and Boundaries](#)
- * [POSIX Classes](#)
- * [Inline Modifiers](#)
- * [Lookarounds](#)
- * [Character Class Operations](#)
- * [Other Syntax](#)

([direct link](#)).

Characters

Character	Legend	Example	Sample Match
-----------	--------	---------	--------------

\d	Most engines: one digit from 0 to 9	file_\d\d	file_25
\d	.NET, Python 3: one Unicode digit in any script	file_\d\d	file_9३
\w	Most engines: "word character": ASCII letter, digit or underscore	\w-\w\w\w	A-b_1
\w	.Python 3: "word character": Unicode letter, ideogram, digit, or underscore	\w-\w\w\w	字-ま_ॢ
\w	.NET: "word character": Unicode letter, ideogram, digit, or connector	\w-\w\w\w	字-ま_ᲃ
\s	Most engines: "whitespace character": space, tab, newline, carriage return, vertical tab	a\s b	a b
\s		c	c
\s	.NET, Python 3, JavaScript: "whitespace character": any Unicode separator	a\s b	a b
\s		c	c
\D	One character that is not a <i>digit</i> as defined by your engine's \d	\D\D\D	ABC
\W	One character that is not a <i>word character</i> as defined by your engine's \w	\W\W\W\W\W	*-+=)
\S	One character that is not a <i>whitespace character</i> as defined by your engine's \s	\S\S\S\S	Yoyo

([direct link](#)).

Quantifiers

Quantifier	Legend	Example	Sample Match
+	One or more	Version \w-\w+	Version A-b1_1
{3}	Exactly three times	\D{3}	ABC
{2,4}	Two to four times	\d{2,4}	156
{3,}	Three or more times	\w{3,}	regex_tutorial
*	Zero or more times	A*B*C*	AAACC
?	Once or none	plurals?	plural

([direct link](#)).

More Characters

Character	Legend	Example	Sample Match
.	Any character except line break	a.c	abc
.	Any character except line break	.*	whatever, man.
\.	A period (special character: needs to be escaped by a \)	a\.c	a.c
\	Escapes a special character	\.*\+ \? \\$\$^\ \ .*\+? \$^^\	
\	Escapes a special character	\[\{\(\)\}\]	[{()}]

([direct link](#)).

Logic

Logic	Legend	Example	Sample Match
	Alternation / OR operand	22 33	33
(...)	Capturing group	A(nt pple)	Apple (captures "pple")
\1	Contents of Group 1	r(\w)g\1x	regex
\2	Contents of Group 2	(\d\d)+(\d\d)=\2+\1	12+65=65+12
(?: ...)	Non-capturing group	A(?:nt pple)	Apple

([direct link](#)).

More White-Space

Character	Legend	Example	Sample Match
\t	Tab	T\tw{2}	T ab
\r	Carriage return character	see below	
\n	Line feed character	see below	
\r\n	Line separator on Windows	AB\r\nCD	AB CD
\N	Perl, PCRE (C, PHP, R...): one character that is not a line break	\N+	ABC
\h	Perl, PCRE (C, PHP, R...), Java: one horizontal whitespace character: tab or Unicode space separator		
\H	One character that is not a horizontal whitespace		
\v	.NET, JavaScript, Python, Ruby: vertical tab		
\V	Perl, PCRE (C, PHP, R...), Java: one vertical whitespace character: line feed, carriage return, vertical tab, form feed, paragraph or line separator		
\V	Perl, PCRE (C, PHP, R...), Java: any character that is not a vertical whitespace		
\R	Perl, PCRE (C, PHP, R...), Java: one line break (carriage return + line feed pair, and all the characters matched by \v)		

[\(direct link\)](#).

More Quantifiers

Quantifier	Legend	Example	Sample Match
+	The + (one or more) is "greedy"	\d+	12345
?	Makes quantifiers "lazy"	\d+?	1 in 12345
*	The * (zero or more) is "greedy"	A*	AAA
?	Makes quantifiers "lazy"	A*?	empty in AAA
{2,4}	Two to four times, "greedy"	\w{2,4}	abcd
?	Makes quantifiers "lazy"	\w{2,4}?	ab in abcd

[\(direct link\)](#).

Character Classes

Character	Legend	Example	Sample Match
[...]	One of the characters in the brackets	[AEIOU]	One uppercase vowel
[...]	One of the characters in the brackets	T[ao]p	<i>Tap</i> or <i>Top</i>
-	Range indicator	[a-z]	One lowercase letter
[x-y]	One of the characters in the range from x to y	[A-Z]+	GREAT
[...]	One of the characters in the brackets	[AB1-5w-z]	One of either: A,B,1,2,3,4,5,w,x,y,z
[x-y]	One of the characters in the range from x to y	[-~]+	Characters in the printable section of the ASCII table .
[^x]	One character that is not x	[^a-z]{3}	A1!
[^x-y]	One of the characters not in the range from x to y	[^ -~]+	Characters that are not in the printable section of the ASCII table .
[\d\D]	One character that is a digit or a non-digit	[\d\D]+	Any characters, including new lines, which the regular dot doesn't match
[\x41]	Matches the character at hexadecimal position 41 in the ASCII table, i.e. A	[\x41-\x45]{3}	ABE

[\(direct link\)](#).

Anchors and Boundaries

Anchor	Legend	Example	Sample Match
--------	--------	---------	--------------

^	<u>Start of string</u> or <u>start of line</u> depending on multiline mode. (But when [^inside brackets], it means "not")	^abc.*	abc (line start)
\$	<u>End of string</u> or <u>end of line</u> depending on multiline mode. Many engine-dependent subtleties.	.*? the end\$	this is the end
\A	<u>Beginning of string</u> (all major engines except JS)	\Aabc[\\d\\D]*	abc (string... ...start)
\\z	<u>Very end of the string</u> Not available in Python and JS	the end\\z	this is...\\n... the end
\\Z	<u>End of string</u> or (except Python) before final line break Not available in JS	the end\\Z	this is...\\n... the end \\n
\\G	<u>Beginning of String or End of Previous Match</u> .NET, Java, PCRE (C, PHP, R...), Perl, Ruby		
\\b	<u>Word boundary</u> Most engines: position where one side only is an ASCII letter, digit or underscore	Bob.*\\bcat\\b	Bob ate the cat
\\b	<u>Word boundary</u> .NET, Java, Python 3, Ruby: position where one side only is a Unicode letter, digit or underscore	Bob.*\\b\\кошка\\b	Bob ate the кошка
\\B	<u>Not a word boundary</u>	c.*\\Bcat\\B.*	copycats

([direct link](#)).

POSIX Classes

Character	Legend	Example	Sample Match
[[:alpha:]]	PCRE (C, PHP, R...): ASCII letters A-Z and a-z	[8[:alpha:]]+	WellDone88
[[:alpha:]]	Ruby 2: Unicode letter or ideogram	[[:alpha:]]+\\d	кошка99
[[:alnum:]]	PCRE (C, PHP, R...): ASCII digits and letters A-Z and a-z	[[:alnum:]]{10}	ABCDE12345
[[:alnum:]]	Ruby 2: Unicode digit, letter or ideogram	[[:alnum:]]{10}	кошка90210
[[:punct:]]	PCRE (C, PHP, R...): ASCII punctuation mark	[[:punct:]]+	?!,,:;
[[:punct:]]	Ruby: Unicode punctuation mark	[[:punct:]]+	?,,:^\\}

([direct link](#)).

Inline Modifiers

None of these are supported in JavaScript. In Ruby, beware of (?s) and (?m).

Modifier	Legend	Example	Sample Match
(?i)	<u>Case-insensitive mode</u> (except JavaScript)	(?i)Monday	monDAY
(?s)	<u>DOTALL mode</u> (except JS and Ruby). The dot (.) matches new line characters (\\r\\n). Also known as "single-line mode" because the dot treats the entire input as a single line	(?s)From A.*to Z	From A to Z
(?m)	<u>Multiline mode</u> (except Ruby and JS) ^ and \$ match at the beginning and end of every line	(?m)1\\r\\n^2\$\\r\\n^3\$	1 2 3
(?m)	<u>In Ruby</u> : the same as (?s) in other engines, i.e. DOTALL mode, i.e. dot matches line breaks	(?m)From A.*to Z	From A to Z
(?x)	<u>Free-Spacing Mode mode</u> (except JavaScript). Also known as comment mode or whitespace mode	(?x) # this is a # comment abc # write on multiple # lines []d # spaces must be # in brackets	abc d
(?n)	<u>.NET, PCRE 10.30+: named capture only</u>	Turns all (parentheses) into non-capture groups. To capture, use <u>named groups</u> .	

(?d)	Java: Unix linebreaks only .	The dot and the ^ and \$ anchors are only affected by \n
(?^)	PCRE 10.32+: unset modifiers	Unsets ismnx modifiers

([direct link](#)).

Lookarounds

Lookaround	Legend	Example	Sample Match
(?=...)	Positive lookahead	(?=\d{10})\d{5}	01234 in 01234 56789
(?<=...)	Positive lookbehind	(?<=\d)cat	cat in 1 cat
(?!...)	Negative lookahead	(?!theatre)the\w+	theme
(?<!...)	Negative lookbehind	\w{3}(?<!mon)ster	Munster

([direct link](#)).

Character Class Operations

Class Operation	Legend	Example	Sample Match
[...-[...]]	.NET: character class subtraction. One character that is in those on the left, but not in the subtracted class.	[a-z-[aeiou]]	Any lowercase consonant
[...-[...]]	.NET: character class subtraction.	[\p{IsArabic}-[\D]]	An Arabic character that is not a non-digit, i.e., an Arabic digit
[...&&[...]]	Java, Ruby 2+: character class intersection. One character that is both in those on the left and in the && class.	[\S&&[\D]]	An non-whitespace character that is a non-digit.
[...&&[...]]	Java, Ruby 2+: character class intersection.	[\S&&[\D]&&[^a-zA-Z]]	An non-whitespace character that a non-digit and not a letter.
[...&&[^...]]	Java, Ruby 2+: character class subtraction is obtained by intersecting a class with a negated class	[a-z&&[^aeiou]]	An English lowercase letter that is not a vowel.
[...&&[^...]]	Java, Ruby 2+: character class subtraction	[\p{InArabic}&&[^\p{L}\p{N}]]	An Arabic character that is not a letter or a number

([direct link](#)).

Other Syntax

Syntax	Legend	Example	Sample Match
\K	Keep Out Perl, PCRE (C, PHP, R...), Python's alternate <i>regex</i> engine, Ruby 2+: drop everything that was matched so far from the overall match to be returned	prefix\K\d+	12
\Q...\E	Perl, PCRE (C, PHP, R...), Java: treat anything between the delimiters as a literal string. Useful to escape metacharacters.	\Q(C++?)\E	(C++ ?)

Don't Miss The [Regex Style Guide](#)

and [The Best Regex Trick Ever!!!](#)



[The 1001 ways to use Regex](#)



Ask Rex



Buy me a coffee



Buy me a coffee

© Copyright RexEgg.com